

SIMATS ENGINEERING

ASSESSMENT TOOL 3

NAME: ADHIL BIJUKUMAR

REG NO: 192411133

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

Question 1

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

Answer:

Scenario: A company wants its website to work seamlessly on mobile, tablet, and desktop devices.

Responsive web design and its importance

Responsive web design (RWD) is an approach where a single website automatically adjusts its layout, images, and content to fit the screen size, resolution, and orientation of whatever device is viewing it — instead of building separate sites for mobile and desktop.

- Reach: one codebase serves phones, tablets, laptops, and large monitors, reducing development and maintenance cost.
- User experience: content stays readable and controls stay tappable/clickable regardless of device, reducing bounce rate.
- SEO: search engines favour mobile-friendly sites and penalise sites that require separate mobile URLs.
- Future-proofing: new device sizes (foldables, large TVs) are accommodated without a redesign.

Key CSS techniques used to achieve responsiveness

- Fluid grids/units — using %, rem, vw/vh instead of fixed px so elements scale with the viewport.
- Flexible images — max-width: 100%; height: auto; so images shrink within their container.
- Media queries — @media (min-width/max-width) apply different rules at defined breakpoints.
- Flexbox and Grid — layout models that reflow content automatically as space changes.
- The viewport meta tag — `<meta name="viewport" content="width=device-width, initial-scale=1">` tells mobile browsers to render at device width instead of a zoomed-out desktop layout.

Using Flexbox / Grid to design adaptive layouts

- **Flexbox** arranges items along a single axis and lets them grow, shrink, or wrap; ideal for nav bars, card rows, and toolbars that must adapt their item count per row as space changes.
- **Grid** defines rows and columns together, so a page skeleton (header/sidebar/main) can be redefined at each breakpoint using grid-template-columns and grid-template-areas without changing the HTML.

Example layout

```
<div class="cards">
  <div class="card">1</div>
  <div class="card">2</div>
  <div class="card">3</div>
</div>

/* Mobile-first base */
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}
.card {
  flex: 1 1 100%; /* one card per row on mobile */
  background: #eef3fa;
  padding: 1rem;
}

/* Tablet */
@media (min-width: 600px) {
  .card { flex: 1 1 45%; } /* two per row */
}

/* Desktop */
@media (min-width: 1024px) {
  .card { flex: 1 1 30%; } /* three per row */
}
```

Suggested improvements for better cross-device UX

- Use relative font units (rem) with a base defined in the root, so text respects user accessibility zoom settings.
- Test touch-target sizes (minimum ~44x44px) on mobile so buttons remain easy to tap.
- Use srcset/sizes on to serve appropriately sized images per device, improving load speed on mobile data.
- Adopt a mobile-first CSS order (base styles for small screens, media queries add complexity for larger ones) to keep mobile performance light.

Question 2

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Answer:

Scenario: A webpage contains a form where user input must be validated before submission.

Client-side validation and its purpose

Client-side validation checks form input in the browser, using JavaScript (or built-in HTML attributes), before the data is sent to the server. Its purpose is to give the user immediate feedback, catch obvious errors early, and reduce unnecessary server round-trips — though it must always be paired with server-side validation, since client-side checks can be bypassed.

Role of JavaScript event handling in validation

- submit — intercepted on the <form> to run all checks and call preventDefault() if any field fails.

- input / change — validate a field as the user types or after they leave it, giving real-time feedback.
- blur — commonly used to validate a single field once the user moves to the next one, avoiding premature error messages while still typing.

Role of regular expressions in validating email/phone inputs

A regular expression (regex) is a pattern that describes the expected shape of valid text. For structured inputs like emails and phone numbers, a regex checks that the value matches the required format (e.g., text@text.domain, or a fixed count of digits) far more reliably than manual character-by-character checks.

Example

```
<form id="signupForm">
  <input type="email" id="email" placeholder="Email">
  <span id="emailError" class="error"></span>

  <input type="tel" id="phone" placeholder="Phone (10 digits)">
  <span id="phoneError" class="error"></span>

  <button type="submit">Submit</button>
</form>

const form = document.getElementById('signupForm');
const email = document.getElementById('email');
const phone = document.getElementById('phone');

const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const phonePattern = /^[0-9]{10}$/;

form.addEventListener('submit', (e) => {
  e.preventDefault();
  let valid = true;

  if (!emailPattern.test(email.value)) {
    document.getElementById('emailError').textContent = 'Enter a valid email';
    valid = false;
  }
})
```

```
if (!phonePattern.test(phone.value)) {  
    document.getElementById('phoneError').textContent = 'Enter a 10-digit phone number';  
    valid = false;  
}  
  
if (valid) form.submit();  
});
```

Improvements for usability and security

- Usability: validate on blur/input (not only on submit) so users see and fix errors before reaching the submit button.
- Usability: clear error messages next to the field rather than a generic alert, and remove the error as soon as the input becomes valid.
- Security: never rely on client-side validation alone — always re-validate and sanitize the same fields on the server, since JavaScript can be disabled or bypassed.
- Security: escape/encode user input before rendering it elsewhere on the page to prevent XSS, even if the format passed validation.

Question 3

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

Answer:

Scenario: A webpage layout breaks when the browser window is resized.

The CSS box model and its components

Every HTML element is rendered as a rectangular box made up of four layers, counted from the inside out:

- Content — the actual text/image, sized by width and height.

- Padding — transparent space between the content and the border; part of the element's clickable/background area.
- Border — a visible (or invisible) line surrounding the padding.
- Margin — transparent space outside the border, separating the element from its neighbours.
- By default, **width/height** apply only to content, so padding and border add on top; setting **box-sizing: border-box**; makes width/height include padding and border, which prevents many resize-related overflow issues.

How CSS positioning techniques affect layout structure

- static — default flow; the element does not respond to top/left/right/bottom.
- relative — stays in normal flow but can be nudged from its original position; also establishes a containing block for absolutely positioned children.
- absolute — removed from normal flow and positioned relative to the nearest positioned ancestor; can overlap other elements if not managed carefully.
- fixed — positioned relative to the viewport, staying in place during scroll (e.g., a fixed header).
- sticky — behaves as relative until a scroll threshold, then acts as fixed within its container.

Possible causes of layout breaking on resize

- Fixed pixel widths on containers or images that don't shrink below the viewport width, forcing horizontal scroll.
- Missing box-sizing: border-box, so padding/border push elements wider than their intended width.
- Absolutely positioned elements with hard-coded coordinates that no longer align once surrounding content reflows.
- No viewport meta tag, causing mobile browsers to render the desktop layout zoomed out.
- Flex/grid items without min-width: 0 or flex-wrap, causing overflow instead of wrapping.

How media queries resolve these issues

Media queries let CSS apply different rules once the viewport crosses defined breakpoints (e.g., max-width: 768px), so a designer can swap fixed widths for percentages, stack a multi-column layout into a single column, or hide/show elements — keeping the layout stable at every size rather than relying on one fixed design stretching or breaking.

```
* { box-sizing: border-box; }

.container { width: 100%; max-width: 1200px; margin: 0 auto; padding: 0 1rem; }

@media (max-width: 768px) {
  .layout { grid-template-columns: 1fr; } /* stack columns */
}
```

Best practices for a stable, responsive layout

- Apply box-sizing: border-box globally to keep width calculations predictable.
- Prefer relative units (% , fr, rem, vw) over fixed px for widths and font sizes.
- Design mobile-first, adding complexity through min-width media queries as space increases.
- Use Flexbox/Grid with wrap/minmax() so content reflows instead of overflowing.
- Always test at multiple breakpoints (and by manually dragging the browser edge) during development, not just at fixed device widths.

Question 4

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

Answer:

Scenario: A website requires dynamic content updates without reloading the entire page.

The Document Object Model (DOM) and its role

The DOM is a tree-structured, in-memory representation of an HTML document, where every tag, attribute, and piece of text becomes a node that can be read or modified programmatically. It acts as the interface between the static HTML file and JavaScript: the browser parses HTML into the DOM, and any change to the DOM is immediately reflected on screen — enabling dynamic, interactive pages without needing to reload from the server.

Using JavaScript to manipulate the DOM for dynamic updates

- Selecting nodes — `document.getElementById()`, `querySelector()`, `querySelectorAll()`.
- Reading/changing content — `element.textContent`, `element.innerHTML`.
- Changing structure — `createElement()`, `appendChild()`, `removeChild()`, `replaceChild()`.
- Changing appearance — `element.style`, or toggling classes with `classList.add/remove/toggle`.
- Responding to interaction — `addEventListener()` ties DOM updates to user actions like clicks or form input.

Example — updating content dynamically

```
<ul id="taskList"></ul>
<input id="taskInput" placeholder="New task">
<button id="addBtn">Add</button>

const list = document.getElementById('taskList');
const input = document.getElementById('taskInput');

document.getElementById('addBtn').addEventListener('click', () => {
  if (!input.value.trim()) return;
```

```
const li = document.createElement('li');
li.textContent = input.value;
list.appendChild(li); // updates the page instantly, no reload

input.value = "";
});
```

Improvements to enhance performance and user experience

- Performance: batch multiple DOM changes (e.g., build items in a DocumentFragment) and insert once, instead of calling appendChild repeatedly, to minimise reflows.
- Performance: cache selected elements in variables rather than re-querying the DOM on every event.
- Performance: use event delegation (one listener on a parent list) instead of attaching a listener to every new item.
- UX: animate new/removed items with a CSS transition so updates feel smooth rather than abrupt.
- UX: show a brief loading or confirmation state when content is fetched asynchronously (e.g., via fetch) so users know an update is in progress.